

INSTITUTO FEDERAL DE PERNAMBUCO CAMPUS BELO JARDIM

CAMILLE MARIA DIAS DE BARROS SILVA
GABRIEL CAMELO
IGOR NAYAN
LUCAS CAVALCANTE DA SILVA
PEDRO VINICIUS SILVA LIRA

Documentação de Casos de Teste
Mais água para nosso povo!

BELO JARDIM, PE
2026

Módulo de Autenticação e Segurança

1. Login com Primeiro Acesso

- **ID do Caso de Teste:** CT-001
- **Nome:** Teste de Login com Primeiro Acesso
- **Objetivo:** Garantir que o usuário seja bloqueado de fazer login sem redefinir a senha no primeiro acesso.
- **Pré-condições:** Usuário cadastrado com a flag de primeiro acesso ativa.
- **Entradas:**
 1. E-mail do usuário
 2. Senha do usuário
- **Passos para Execução:**
 1. Acessar o endpoint de login.
 2. Enviar os dados de autenticação.
 3. Verificar a resposta do servidor.
- **Resultado Esperado:** O sistema deve retornar status HTTP 403 e uma mensagem orientando a troca de senha.
- **Critérios de Sucesso:** Retorno do código 403 e da mensagem de instrução.
- **Critérios de Falha:** O login é concluído com sucesso ou outro erro genérico é retornado.

2. Login com Credenciais Válidas

- **ID do Caso de Teste:** CT-002
- **Nome:** Teste de Login com Credenciais Válidas
- **Objetivo:** Garantir que o usuário consiga fazer login com credenciais corretas e receba um token.
- **Pré-condições:** Usuário cadastrado e sem flag de primeiro acesso.
- **Entradas:**
 1. E-mail do usuário
 2. Senha do usuário

- **Passos para Execução:**

1. Acessar o endpoint de login.
2. Enviar os dados de autenticação.
3. Verificar a resposta do servidor.

- **Resultado Esperado:** O sistema deve retornar status HTTP 200 e o Token de autenticação.

- **Critérios de Sucesso:** O token JWT é retornado.

- **Critérios de Falha:** O login falha ou nenhum token é gerado.

3. Alteração de Senha do Usuário

- **ID do Caso de Teste:** CT-003

- **Nome:** Teste de Alteração de Senha do Usuário

- **Objetivo:** Garantir que o usuário consiga alterar sua senha e receba um novo token.

- **Pré-condições:** Usuário cadastrado e logado no sistema.

- **Entradas:**

1. ID do usuário
2. Nova senha

- **Passos para Execução:**

1. Acessar o endpoint de alteração de senha.
2. Enviar a nova senha.
3. Verificar a resposta do servidor.

- **Resultado Esperado:** O sistema deve retornar status HTTP 200 e um novo Token de autenticação.

- **Critérios de Sucesso:** A senha é atualizada e o token é retornado.

- **Critérios de Falha:** A senha não é atualizada ou a requisição falha.

4. Atualização de Senha do Usuário

- **ID do Caso de Teste:** CT-004
- **Nome:** Teste de Atualização de Senha do Usuário
- **Objetivo:** Garantir que a atualização da senha mediante validação da antiga seja processada com sucesso.
- **Pré-condições:** Usuário cadastrado e logado.
- **Entradas:**

1. ID do usuário

2. Senha atual

3. Nova senha

- **Passos para Execução:**

1. Acessar o endpoint de atualização de senha.

2. Enviar a senha atual e a nova senha.

3. Verificar a resposta do servidor.

- **Resultado Esperado:** O sistema deve retornar status HTTP 200 e um novo Token JWT.
- **Critérios de Sucesso:** A senha é validada, atualizada e o token é gerado.
- **Critérios de Falha:** Erro de validação ou falha na atualização.

5. Geração e Extração de Token JWT

- **ID do Caso de Teste:** CT-005
- **Nome:** Teste de Geração e Extração de Token JWT
- **Objetivo:** Garantir que a geração do token e a posterior extração do e-mail funcionem corretamente.
- **Pré-condições:** Os dados do usuário estão instanciados em memória.
- **Entradas:**

1. Usuário com E-mail e Roles definidas

- **Passos para Execução:**

1. Acionar o serviço de geração de JWT.

2. Solicitar a extração do e-mail do token gerado.

3. Validar o e-mail retornado.

- **Resultado Esperado:** O token deve ser gerado e o e-mail extraído deve corresponder ao e-mail do usuário.

- **Critérios de Sucesso:** Geração do token e validação do e-mail extraído.

- **Critérios de Falha:** Falha na geração ou e-mail extraído incorreto.

6. Validação de Token JWT Correto

- **ID do Caso de Teste:** CT-006

- **Nome:** Teste de Validação de Token JWT Correto

- **Objetivo:** Garantir que um token válido seja reconhecido como autêntico pelo sistema.

- **Pré-condições:** Um token JWT recém-gerado em memória para o usuário.

- **Entradas:**

1. Token JWT

2. UserDetails do usuário

- **Passos para Execução:**

1. Acionar o serviço de validação de token.

2. Verificar o retorno do método.

- **Resultado Esperado:** O sistema deve retornar verdadeiro (true) para a validação.

- **Critérios de Sucesso:** A validação é bem-sucedida.

- **Critérios de Falha:** O sistema rejeita o token autêntico.

7. Rejeição de Token JWT com E-mail Incorreto

- **ID do Caso de Teste:** CT-007
- **Nome:** Teste de Rejeição de Token JWT com E-mail Incorreto
- **Objetivo:** Garantir que o token seja rejeitado caso não pertença ao usuário informado.
- **Pré-condições:** Token gerado para um e-mail específico.
- **Entradas:**

1. Token JWT

2. UserDetails com e-mail diferente

- **Passos para Execução:**

1. Acionar o serviço de validação.

2. Verificar o retorno do método.

- **Resultado Esperado:** O sistema deve retornar falso (false) indicando invalidez.
- **Critérios de Sucesso:** A validação falha corretamente.
- **Critérios de Falha:** O token de outro usuário é aceito como válido.

8. Validação de Token JWT Expirado

- **ID do Caso de Teste:** CT-008
- **Nome:** Teste de Validação de Token JWT Expirado
- **Objetivo:** Garantir que a tentativa de validar um token expirado resulte em uma exceção.
- **Pré-condições:** O serviço JWT configurado com tempo de expiração negativo.
- **Entradas:**

1. Token JWT expirado

2. UserDetails do usuário

- **Passos para Execução:**

1. Acionar a validação do token.

2. Monitorar a exceção gerada.

- **Resultado Esperado:** O sistema deve lançar a exceção `ExpiredJwtException`.
- **CrITÉrios de Sucesso:** A exceção de expiração é lançada adequadamente.
- **CrITÉrios de Falha:** Nenhuma exceção é lançada ou o token é considerado válido.

Módulo de Gestão de Usuários e Cargos

9. Criação de Novo Usuário (Controlador)

- **ID do Caso de Teste:** CT-009
- **Nome:** Teste de Criação de Novo Usuário (Controlador)
- **Objetivo:** Garantir que o endpoint processe corretamente a criação de um usuário.
- **Pré-condições:** O sistema e o banco de dados estão acessíveis.
- **Entradas:**

1. Dados do novo usuário (nome, e-mail, senha, ID da Role)

- **Passos para Execução:**

1. Acessar o endpoint POST de usuários.

2. Enviar o payload com os dados.

3. Verificar o código de status retornado.

- **Resultado Esperado:** O sistema deve retornar status HTTP 201 e os dados básicos do usuário.
- **CrITÉrios de Sucesso:** O usuário é criado com o ID esperado.
- **CrITÉrios de Falha:** O retorno possui status de erro ou a criação falha.

10. Atualização de Usuário Existente (Controlador)

- **ID do Caso de Teste:** CT-010
- **Nome:** Teste de Atualização de Usuário Existente (Controlador)
- **Objetivo:** Garantir que a atualização dos dados do usuário ocorra corretamente pela API.
- **Pré-condições:** O usuário existe no banco de dados.
- **Entradas:**

1. ID do usuário

2. Dados atualizados (nome, e-mail)

- **Passos para Execução:**

1. Acessar o endpoint PUT de usuários com o ID.

2. Enviar o payload.

3. Verificar o código retornado.

- **Resultado Esperado:** O sistema deve retornar status HTTP 200 e os dados atualizados.
- **CrITÉrios de Sucesso:** A atualização reflete no retorno da requisição.
- **CrITÉrios de Falha:** Os dados não são alterados ou ocorre erro.

11. Listagem de Todos os Usuários (Gestão)

- **ID do Caso de Teste:** CT-011
- **Nome:** Teste de Listagem de Todos os Usuários (Gestão)
- **Objetivo:** Garantir que a listagem de usuários retorne a coleção esperada.
- **Pré-condições:** Usuários previamente cadastrados.
- **Entradas:**

1. Nenhuma

- **Passos para Execução:**

1. Acessar o endpoint GET de listagem de usuários.

2. Verificar o payload retornado.

- **Resultado Esperado:** O sistema deve retornar status HTTP 200 contendo a lista de usuários.

- **CrITÉrios de Sucesso:** A lista de usuários é exibida no JSON de resposta.

- **CrITÉrios de Falha:** O retorno é vazio ou gera erro.

12. Listagem de Todos os Cargos

- **ID do Caso de Teste:** CT-012

- **Nome:** Teste de Listagem de Todos os Cargos

- **Objetivo:** Garantir que a listagem de cargos retorne a coleção esperada.

- **Pré-condições:** Cargos previamente cadastrados.

- **Entradas:**

1. Nenhuma

- **Passos para Execução:**

1. Acessar o endpoint GET de roles.

2. Verificar os cargos retornados.

- **Resultado Esperado:** O sistema deve retornar status HTTP 200 contendo a lista de cargos.

- **CrITÉrios de Sucesso:** A lista de cargos é recebida com sucesso.

- **CrITÉrios de Falha:** Ocorre falha na listagem.

13. Listagem de Todas as Permissões

- **ID do Caso de Teste:** CT-013
- **Nome:** Teste de Listagem de Todas as Permissões
- **Objetivo:** Garantir que a listagem de permissões funcione corretamente.
- **Pré-condições:** Permissões previamente cadastradas.
- **Entradas:**

1. Nenhuma

- **Passos para Execução:**

1. Acessar o endpoint GET de permissions.

2. Verificar as permissões retornadas.

- **Resultado Esperado:** O sistema deve retornar status HTTP 200 contendo as permissões.
- **CrITÉrios de Sucesso:** As permissões são retornadas no formato JSON.
- **CrITÉrios de Falha:** O endpoint retorna erro.

14. Criação de Cargo

- **ID do Caso de Teste:** CT-014
- **Nome:** Teste de Criação de Cargo
- **Objetivo:** Garantir que o sistema permita o cadastro de novos cargos.
- **Pré-condições:** Nenhuma.
- **Entradas:**

1. Dados do novo cargo (nome, lista de permissões)

- **Passos para Execução:**

1. Acessar o endpoint POST de criação de cargo.

2. Enviar o payload de cargo.

3. Verificar o status e resposta.

- **Resultado Esperado:** O sistema deve retornar status HTTP 201 e os dados do cargo.

- **Critérios de Sucesso:** O cargo é criado e retornado com o ID.

- **Critérios de Falha:** Falha na criação ou retorno incorreto.

15. Atualização de Cargo

- **ID do Caso de Teste:** CT-015

- **Nome:** Teste de Atualização de Cargo

- **Objetivo:** Garantir que um cargo existente possa ser atualizado.

- **Pré-condições:** Cargo previamente cadastrado.

- **Entradas:**

1. ID do cargo

2. Novos dados do cargo

- **Passos para Execução:**

1. Acessar o endpoint PUT de atualização de cargo.

2. Enviar os novos dados.

3. Verificar o resultado.

- **Resultado Esperado:** O sistema deve retornar status HTTP 200 com os dados atualizados.

- **Critérios de Sucesso:** Os dados do cargo são salvos e retornados.

- **Critérios de Falha:** Os dados não atualizam ou geram exceção.

16. Exclusão de Cargo

- **ID do Caso de Teste:** CT-016
- **Nome:** Teste de Exclusão de Cargo
- **Objetivo:** Garantir que a exclusão de cargo via API funcione corretamente.
- **Pré-condições:** O cargo existe e não possui vínculos obrigatórios.
- **Entradas:**

1. ID do cargo a excluir

- **Passos para Execução:**

1. Acessar o endpoint DELETE de cargos.

2. Enviar a requisição de exclusão.

3. Verificar o status retornado.

- **Resultado Esperado:** O sistema deve retornar status HTTP 204 No Content.
- **Critérios de Sucesso:** O retorno 204 é recebido.
- **Critérios de Falha:** Erro ou status de conflito.

17. Recuperação de Lista de RoleDTO

- **ID do Caso de Teste:** CT-017
- **Nome:** Teste de Recuperação de Lista de RoleDTO
- **Objetivo:** Garantir que o serviço de cargo consiga mapear e retornar uma lista de DTOs.
- **Pré-condições:** Existem cargos no repositório.
- **Entradas:**

1. Nenhuma

- **Passos para Execução:**

1. Chamar o método findAllRoles do RoleService.

2. Validar a lista gerada.

- **Resultado Esperado:** A lista contendo RoleDTO deve ser retornada.
- **Critérios de Sucesso:** A lista de RoleDTO contém os dados corretos.
- **Critérios de Falha:** Falha no mapeamento ou lista vazia incorreta.

18. Busca de Cargo por ID Válido

- **ID do Caso de Teste:** CT-018
- **Nome:** Teste de Busca de Cargo por ID Válido
- **Objetivo:** Garantir que a busca interna encontre o cargo correto.
- **Pré-condições:** O cargo com o ID buscado existe.
- **Entradas:**

1. ID do cargo (1L)

- **Passos para Execução:**

1. Chamar o método findById do RoleService.

2. Validar a entidade retornada.

- **Resultado Esperado:** A entidade Role deve ser retornada perfeitamente preenchida.
- **Critérios de Sucesso:** O cargo é retornado e possui o nome correto.
- **Critérios de Falha:** Exceção não tratada ou entidade nula.

19. Exceção ao Buscar Cargo Inexistente

- **ID do Caso de Teste:** CT-019
- **Nome:** Teste de Exceção ao Buscar Cargo Inexistente
- **Objetivo:** Garantir que a busca de cargo inexistente gere EntityNotFoundException.
- **Pré-condições:** O cargo não existe no repositório.
- **Entradas:**

1. ID do cargo (Inexistente)

- **Passos para Execução:**

1. Chamar o método `findByld` do `RoleService`.

2. Aguardar a exceção.

- **Resultado Esperado:** Deve lançar `EntityNotFoundException`.
- **CrITÉRIOS de Sucesso:** A exceção esperada é lançada.
- **CrITÉRIOS de Falha:** Nenhuma exceção ocorre ou exceção genérica é disparada.

20. Salvar Novo Cargo com Permissões

- **ID do Caso de Teste:** CT-020

- **Nome:** Teste de Salvar Novo Cargo com Permissões

- **Objetivo:** Garantir que a lógica de salvamento associe corretamente as permissões.

- **Pré-condições:** Permissões existem no sistema.

- **Entradas:**

1. `RoleDTO` com ID nulo

2. Permissões DTO

- **Passos para Execução:**

1. Chamar o método `saveRole`.

2. Validar o salvamento das entidades.

- **Resultado Esperado:** O cargo deve ser salvo contendo as permissões corretas.
- **CrITÉRIOS de Sucesso:** O `RoleDTO` retornado contém o nome esperado.
- **CrITÉRIOS de Falha:** Erro no salvamento da relação.

21. Exceção ao Salvar Cargo com Permissão Inválida

- **ID do Caso de Teste:** CT-021
- **Nome:** Teste de Exceção ao Salvar Cargo com Permissão Inválida
- **Objetivo:** Garantir que associar permissões inexistentes gere erro.
- **Pré-condições:** As permissões informadas não existem.
- **Entradas:**

1. RoleDTO contendo permissão inexistente

- **Passos para Execução:**

1. Chamar o método saveRole.

2. Aguardar a exceção.

- **Resultado Esperado:** Deve lançar EntityNotFoundException.
- **CrITÉRIOS de Sucesso:** A exceção de não encontrado é ativada.
- **CrITÉRIOS de Falha:** O sistema salva silenciosamente o cargo.

22. Exclusão de Cargo Sem Usuários Vinculados

- **ID do Caso de Teste:** CT-022
- **Nome:** Teste de Exclusão de Cargo Sem Usuários Vinculados
- **Objetivo:** Garantir que a exclusão direta no serviço ocorra se não houver vínculos.
- **Pré-condições:** O cargo não tem usuários.
- **Entradas:**

1. ID do cargo

- **Passos para Execução:**

1. Chamar o método deleteRole do RoleService.

2. Verificar a deleção no repositório.

- **Resultado Esperado:** A função delete do repositório deve ser invocada 1 vez.
- **Critérios de Sucesso:** O processo de deleção é concluído.
- **Critérios de Falha:** O cargo não é deletado.

23. Exceção ao Excluir Cargo Com Usuários Vinculados

- **ID do Caso de Teste:** CT-023
- **Nome:** Teste de Exceção ao Excluir Cargo Com Usuários Vinculados
- **Objetivo:** Garantir que a exclusão seja evitada caso usuários usem o cargo.
- **Pré-condições:** O cargo possui usuários associados.
- **Entradas:**

1. ID do cargo

- **Passos para Execução:**

1. Chamar o método deleteRole do RoleService.

2. Aguardar a exceção.

- **Resultado Esperado:** Deve lançar IllegalStateException e não deletar o cargo.
- **Critérios de Sucesso:** A exclusão é interceptada corretamente.
- **Critérios de Falha:** O cargo é deletado indevidamente.

24. Atualização de Usuário via Serviço

- **ID do Caso de Teste:** CT-024
- **Nome:** Teste de Atualização de Usuário via Serviço
- **Objetivo:** Garantir que a atualização do nome e e-mail via serviço funcione.
- **Pré-condições:** Usuário existente na base.
- **Entradas:**

1. ID do usuário**2. Dados de UpdateUserDTO****• Passos para Execução:****1. Chamar o método updateUser no UserService.****2. Verificar o mapeamento do retorno.**

- **Resultado Esperado:** Os campos modificados devem refletir na entidade.
- **Critérios de Sucesso:** A entidade e o DTO atualizam com sucesso.
- **Critérios de Falha:** Falha de banco de dados ou erro de lógica.

25. Exceção na Atualização de Usuário Inexistente

- **ID do Caso de Teste:** CT-025
- **Nome:** Teste de Exceção na Atualização de Usuário Inexistente
- **Objetivo:** Garantir que a atualização bloqueie IDs inválidos.
- **Pré-condições:** Usuário não cadastrado.
- **Entradas:**

1. ID inexistente**2. UpdateUserDTO****• Passos para Execução:****1. Chamar o método updateUser.****2. Esperar exceção.**

- **Resultado Esperado:** Deve lançar EntityNotFoundException.
- **Critérios de Sucesso:** A exceção bloqueia o fluxo incorreto.
- **Critérios de Falha:** O fluxo continua ou ocorre um erro nulo.

26. Listagem de Todos os Usuários no Serviço

- **ID do Caso de Teste:** CT-026
- **Nome:** Teste de Listagem de Todos os Usuários no Serviço
- **Objetivo:** Garantir que o UserService acesse o repositório e converta usuários para DTO.
- **Pré-condições:** Existem registros de usuários.
- **Entradas:**

1. Nenhuma

- **Passos para Execução:**

1. Chamar o método findAllUsers.

2. Validar a lista gerada.

- **Resultado Esperado:** Uma lista de UserDTO deve ser devolvida.
- **CrITÉrios de Sucesso:** A lista corresponde aos usuários cadastrados.
- **CrITÉrios de Falha:** Falha no parser ou conexão.

27. Salvamento de Novo Usuário no Serviço

- **ID do Caso de Teste:** CT-027
- **Nome:** Teste de Salvamento de Novo Usuário no Serviço
- **Objetivo:** Garantir que o cadastro salve corretamente a senha codificada e os dados.
- **Pré-condições:** Cargo atribuído ao usuário deve existir.
- **Entradas:**

1. UserDTO com novos dados

- **Passos para Execução:**

1. Chamar o método saveUser.

2. Verificar o repositório.

- **Resultado Esperado:** A entidade User deve ser armazenada e o DTO de resposta criado.
- **Critérios de Sucesso:** O ID e nome correspondem ao novo usuário.
- **Critérios de Falha:** O usuário não é salvo no banco de dados.

Módulo de Gestão de Famílias

28. Criação de Nova Família (Controlador)

- **ID do Caso de Teste:** CT-028
- **Nome:** Teste de Criação de Nova Família (Controlador)
- **Objetivo:** Garantir que a API consiga persistir uma família, incluindo membros e cisternas.
- **Pré-condições:** Não aplicável.
- **Entradas:**

1. Dados da família

2. Lista de membros

3. Lista de cisternas

- **Passos para Execução:**

1. Enviar requisição POST para endpoint de famílias.

2. Validar a persistência.

- **Resultado Esperado:** Retornar HTTP 201 Created e o ID da família.
- **Critérios de Sucesso:** Família salva corretamente com todos os dados atrelados.
- **Critérios de Falha:** Erro ao serializar DTOs ou erro interno 500.

29. Atualização de Família (Controlador)

- **ID do Caso de Teste:** CT-029
- **Nome:** Teste de Atualização de Família (Controlador)
- **Objetivo:** Garantir que a atualização de registros modifique os dados da família.
- **Pré-condições:** Família cadastrada com o ID informado.
- **Entradas:**

1. ID da família

2. Novos dados no payload PUT

- **Passos para Execução:**

1. Enviar requisição PUT.

2. Aguardar confirmação.

- **Resultado Esperado:** Retornar HTTP 200 OK com dados atualizados.
- **Critérios de Sucesso:** A alteração afeta os campos no retorno JSON.
- **Critérios de Falha:** Falha na validação de ID.

30. Busca de Família por ID (Controlador)

- **ID do Caso de Teste:** CT-030
- **Nome:** Teste de Busca de Família por ID (Controlador)
- **Objetivo:** Garantir que a API recupere os dados da família correta.
- **Pré-condições:** A família alvo existe.
- **Entradas:**

1. ID da família

- **Passos para Execução:**

1. Enviar requisição GET por ID.**2. Ler a resposta da API.**

- **Resultado Esperado:** Retornar HTTP 200 OK contendo os detalhes.
- **Critérios de Sucesso:** A resposta reflete os dados da entidade solicitada.
- **Critérios de Falha:** Família não encontrada ou ID trocado.

31. Listagem Paginada de Todas as Famílias (Controlador)

- **ID do Caso de Teste:** CT-031
- **Nome:** Teste de Listagem Paginada de Todas as Famílias (Controlador)
- **Objetivo:** Garantir que a API possa exportar dados paginados.
- **Pré-condições:** Existem dezenas de famílias na base.
- **Entradas:**

1. Parâmetros de paginação padrão

- **Passos para Execução:**

1. Enviar requisição GET para listagem de famílias.

- **Resultado Esperado:** Retornar HTTP 200 OK contendo array `content` e metadados de paginação.
- **Critérios de Sucesso:** O primeiro item deve possuir o nome esperado e a estrutura de página ser válida.
- **Critérios de Falha:** A página vem vazia indevidamente.

32. Filtragem de Famílias por Nome

- **ID do Caso de Teste:** CT-032
- **Nome:** Teste de Filtragem de Famílias por Nome
- **Objetivo:** Garantir que a busca com filtro de nome retorne as correspondências.
- **Pré-condições:** Existem registros com nomes variados.
- **Entradas:**

1. Parâmetro `name` de filtro

- **Passos para Execução:**

1. Enviar GET contendo a query string `name`.

- **Resultado Esperado:** Retornar HTTP 200 OK com resultados filtrados.
- **CrITÉrios de Sucesso:** Todos os resultados possuem o nome condizente com a query.
- **CrITÉrios de Falha:** Os resultados não condizem com a busca.

33. Filtragem de FamÍlias por Status

- **ID do Caso de Teste:** CT-033
- **Nome:** Teste de Filtragem de FamÍlias por Status
- **Objetivo:** Garantir que o filtro por status restrinja corretamente a listagem.
- **Pré-condiçÕes:** FamÍlias marcadas com status específicos (ex: URGENT).
- **Entradas:**

1. Parâmetro `status`

- **Passos para Execução:**

1. Enviar GET contendo query string `status`.

- **Resultado Esperado:** Retornar HTTP 200 OK contendo as famÍlias correspondentes ao enumerador.
- **CrITÉrios de Sucesso:** A resposta mostra as famÍlias do status desejado.
- **CrITÉrios de Falha:** Enum type mismatch ou filtro falho.

34. Retorno de Família Existente (Serviço)

- **ID do Caso de Teste:** CT-034
- **Nome:** Teste de Retorno de Família Existente (Serviço)
- **Objetivo:** Verificar o mapeamento do Service para famÍlias existentes.
- **Pré-condiçÕes:** A Família está salva.
- **Entradas:**

1. ID interno

- **Passos para Execução:**

1. Chamar `getFamilyById`.

- **Resultado Esperado:** Retornar uma entidade de Família mapeada corretamente.
- **Critérios de Sucesso:** A família existe no repositório e é validada.
- **Critérios de Falha:** Erro de busca do repositório.

35. Exceção ao Buscar Família Inexistente (Serviço)

- **ID do Caso de Teste:** CT-035
- **Nome:** Teste de Exceção ao Buscar Família Inexistente (Serviço)
- **Objetivo:** Verificar o lançamento de exceção ao consultar Família que não existe.
- **Pré-condições:** Família deletada ou inexistente.
- **Entradas:**

1. ID inexistente

- **Passos para Execução:**

1. Chamar `getFamilyById`.

- **Resultado Esperado:** Lançar EntityNotFoundException.
- **Critérios de Sucesso:** A exceção previne manipulação indesejada.
- **Critérios de Falha:** Nulo ou erro genérico.

36. Persistência de Família com Relações

- **ID do Caso de Teste:** CT-036
- **Nome:** Teste de Persistência de Família com Relações
- **Objetivo:** Assegurar que a FamilyService salve a família conectada a seus dependentes (membros e cisternas).
- **Pré-condições:** Família nova em DTO.
- **Entradas:**

1. FamilyDTO contendo dependências

- **Passos para Execução:**

1. Chamar ``saveFamily``.

- **Resultado Esperado:** O DTO retornado deve ter os IDs atualizados e relacionamentos instanciados.
- **CrITÉrios de Sucesso:** Cisternas e Membros recebem a referência correta.
- **CrITÉrios de Falha:** Os relacionamentos ficam órfãos.

37. Substituição de Dados, Membros e Cisternas

- **ID do Caso de Teste:** CT-037
- **Nome:** Teste de Substituição de Dados, Membros e Cisternas
- **Objetivo:** Garantir a edição completa do conjunto de dados de uma família.
- **Pré-condições:** Família já previamente populada.
- **Entradas:**

1. FamilyDTO com modificações severas nas listas

- **Passos para Execução:**

1. Chamar ``updateFamily`` no Serviço.

- **Resultado Esperado:** As listas de membros e cisternas anteriores são atualizadas.
- **CrITÉrios de Sucesso:** Reflete no repositório todas as mudanças.
- **CrITÉrios de Falha:** Ocorre duplicação de dados nas listas.

38. Exceção de Atualização em Família Inexistente

- **ID do Caso de Teste:** CT-038
- **Nome:** Teste de Exceção de Atualização em Família Inexistente
- **Objetivo:** Validar a proteção do serviço contra atualizações falhas.
- **Pré-condições:** ID inválido fornecido ao serviço.
- **Entradas:**

1. ID fantasma

2. DTO novo

- **Passos para Execução:**

- 1. Executar ``updateFamily``.

- **Resultado Esperado:** Lançar `EntityNotFoundException`.
 - **CrITÉrios de Sucesso:** Exceção lançada corretamente.
 - **CrITÉrios de Falha:** Operação não travada.

39. Mapeamento de Família com Consumo e Dias Restantes

- **ID do Caso de Teste:** CT-039
- **Nome:** Teste de Mapeamento de Família com Consumo e Dias Restantes
- **Objetivo:** Garantir que cálculos de água e entrega sejam injetados ao buscar detalhes de família.
- **Pré-condições:** Família com cisternas registradas.
- **Entradas:**

1. ID da família

- **Passos para Execução:**

- 1. Executar ``getFamilyDetails``.

- **Resultado Esperado:** `FamilyDTO` é devolvido contendo dados customizados.
 - **CrITÉrios de Sucesso:** Dados contêm as variáveis computadas de consumo e entrega.
 - **CrITÉrios de Falha:** Erro de `Null Pointer` no cálculo.

40. Mapeamento da Paginação de Famílias (Find All)

- **ID do Caso de Teste:** CT-040
- **Nome:** Teste de Mapeamento da Paginação de Famílias (Find All)
- **Objetivo:** Validar que a conversão `Page<Family>` para `Page<FamilyDTO>` ocorra perfeitamente.

- **Pré-condições:** Consulta completa na base.
- **Entradas:**

1. Pageable config

- **Passos para Execução:**

1. Chamar `findAllFamilies`.

- **Resultado Esperado:** Uma instância de página contendo DTOs.
- **Critérios de Sucesso:** Página convertida mantendo metadados (total, tamanho).
- **Critérios de Falha:** Página inválida.

41. Mapeamento de Filtragem por Nome em Serviço

- **ID do Caso de Teste:** CT-041
- **Nome:** Teste de Mapeamento de Filtragem por Nome em Serviço
- **Objetivo:** Validar a conversão da filtragem em DTO.
- **Pré-condições:** Busca no repositório por string.
- **Entradas:**

1. String do nome

2. Pageable

- **Passos para Execução:**

1. Chamar `findFamiliesByName`.

- **Resultado Esperado:** Retorna a página de resultados em DTO.
- **Critérios de Sucesso:** Listagem e mapeamento simultâneos em sucesso.
- **Critérios de Falha:** Falha de stream/map.

42. Mapeamento de Filtragem por Status em Serviço

- **ID do Caso de Teste:** CT-042
- **Nome:** Teste de Mapeamento de Filtragem por Status em Serviço
- **Objetivo:** Validar a conversão em busca de enum.

- **Pré-condições:** Busca via repositório de status.

- **Entradas:**

1. Enum FamilyStatus

2. Pageable

- **Passos para Execução:**

1. Chamar ``findFamiliesByStatus``.

- **Resultado Esperado:** Retorna a página de resultados convertida em DTO.
- **Crerios de Sucesso:** A página é montada corretamente.
- **Crerios de Falha:** Falha na Query.

43. Ordenação de Famílias por Nível de Cisterna (Ascendente)

- **ID do Caso de Teste:** CT-043
- **Nome:** Teste de Ordenação de Famílias por Nível de Cisterna (Ascendente)
- **Objetivo:** Certificar que a listagem atende à ordenação de nível das cisternas.
- **Pré-condições:** Múltiplas famílias com volumes diferentes.
- **Entradas:**

1. Pageable customizado

- **Passos para Execução:**

1. Invocar ``findAllOrderByCisternLevelAsc``.

- **Resultado Esperado:** Retornar a lista em ordem crescente.
- **Crerios de Sucesso:** O menor nível aparece no topo.
- **Crerios de Falha:** Ordenação incorreta.

44. Ordenação de Famílias por Nível de Cisterna (Descendente)

- **ID do Caso de Teste:** CT-044
- **Nome:** Teste de Ordenação de Famílias por Nível de Cisterna (Descendente)
- **Objetivo:** Certificar que a listagem atende à ordenação reversa de nível.

- **Pré-condições:** Múltiplas famílias com volumes diferentes.
- **Entradas:**

1. Pageable customizado

- **Passos para Execução:**

1. Invocar `findAllOrderByCisternLevelDesc``.

- **Resultado Esperado:** Retornar a lista em ordem decrescente.
- **CrITÉrios de Sucesso:** O maior nível aparece no topo.
- **CrITÉrios de Falha:** Ordenação incorreta.

45. Consumo Diário de Água das Famílias

- **ID do Caso de Teste:** CT-045
- **Nome:** Teste de Consumo Diário de Água das Famílias
- **Objetivo:** Testar o evento agendado de redução diária dos níveis das cisternas de todas as famílias.
- **Pré-condições:** Configuração diária definida; famílias possuem cisternas.
- **Entradas:**

1. Consumo base global

- **Passos para Execução:**

1. Executar `consumeWaterFromCisterns`` no serviço da família.

- **Resultado Esperado:** Todas as famílias têm seus volumes subtraídos de acordo com o consumo e as instâncias salvas no repositório.
- **CrITÉrios de Sucesso:** Atualização de todas as cisternas no banco ocorre perfeitamente.
- **CrITÉrios de Falha:** O loop falha ou alguma família não é atualizada.

46. Cálculo de Dias Restantes com Base no Consumo

- **ID do Caso de Teste:** CT-046
- **Nome:** Teste de Cálculo de Dias Restantes com Base no Consumo
- **Objetivo:** Verificar a aritmética que prevê os dias de autonomia.

- **Pré-condições:** Água armazenada total calculada.

- **Entradas:**

1. Volume de Água Total
2. Consumo Diário base

- **Passos para Execução:**

1. Chamar ``calculateRemainingDays``.

- **Resultado Esperado:** O total de dias deve ser a divisão exata com arredondamento estipulado.
- **CrITÉrios de Sucesso:** Retorno inteiro consistente.
- **CrITÉrios de Falha:** Erro de divisão por zero ou arredondamento incorreto.

47. Exceção Quando Consumo Diário é Zero

- **ID do Caso de Teste:** CT-047

- **Nome:** Teste de Exceção Quando Consumo Diário é Zero

- **Objetivo:** Garantir que o sistema previne problemas matemáticos.

- **Pré-condições:** Falha na configuração que resulta em consumo zerado.

- **Entradas:**

1. Volume X
2. Consumo = 0

- **Passos para Execução:**

1. Chamar ``calculateRemainingDays`` com consumo zero.

- **Resultado Esperado:** Lançar `IllegalStateException`.
- **CrITÉrios de Sucesso:** A exceção alerta o erro e trava a execução.
- **CrITÉrios de Falha:** Divisão por zero não tratada (`ArithmeticException`).

Módulo de Cisternas

48. Enchimento de Cisterna Parcial com Resto Zero

- **ID do Caso de Teste:** CT-048
- **Nome:** Teste de Enchimento de Cisterna Parcial com Resto Zero
- **Objetivo:** Garantir a lógica de distribuição de volume nas cisternas disponíveis.
- **Pré-condições:** Família possui cisternas sem lotação máxima.
- **Entradas:**

1. Cisterna da família

2. Quantidade de água

3. Consumo base diário

- **Passos para Execução:**

1. Injetar a água na rotina `fillCisterns`.

2. Avaliar retorno.

- **Resultado Esperado:** A água é adicionada na cisterna e o resto deve retornar zero.
- **Critérios de Sucesso:** A cisterna acumula a litragem e repo salva as alterações.
- **Critérios de Falha:** Sobrecarga de capacidade de armazenamento não prevenida.

49. Redução do Nível das Cisternas

- **ID do Caso de Teste:** CT-049
- **Nome:** Teste de Redução do Nível das Cisternas
- **Objetivo:** Certificar a dedução em litragem conforme o consumo projetado diário da família.
- **Pré-condições:** A cisterna tem mais litros que o consumo subtraído.
- **Entradas:**

1. Cisterna da família

2. Consumo Diário da família

• Passos para Execução:

1. Executar ``consumeDailyWater``.

2. Acompanhar nível das cisternas.

- **Resultado Esperado:** A capacidade deve reduzir proporcionalmente ao gasto da família.
- **Crítérios de Sucesso:** Valor residual subtraído perfeitamente.
- **Crítérios de Falha:** Volume reduz abaixo de zero sem alerta.

50. Cálculo Arredondado para Baixo de Dias Restantes (Cisterna)

- **ID do Caso de Teste:** CT-050
- **Nome:** Teste de Cálculo Arredondado para Baixo de Dias Restantes (Cisterna)
- **Objetivo:** Certificar aritmética isolada do serviço de cisterna.
- **Pré-condições:** Água armazenada vs Consumo diário familiar.
- **Entradas:**

1. Água informada

2. Consumo projetado

• Passos para Execução:

1. Utilizar ``calculateRemainingDays`` do `CisternService`.

- **Resultado Esperado:** O sistema provê o número de dias inteiro com precisão.
- **Crítérios de Sucesso:** A precisão aritmética e o retorno de 3 dias base funcionam.
- **Crítérios de Falha:** Cálculo flutuante inconsistente.

51. Exceção para Consumo Zerado nas Cisternas

- **ID do Caso de Teste:** CT-051
- **Nome:** Teste de Exceção para Consumo Zerado nas Cisternas
- **Objetivo:** Garantir validação similar na classe isolada de cisternas.
- **Pré-condições:** Tentativa de projeção de dias de consumo nulo.
- **Entradas:**
 1. Consumo diário = 0
 2. Volume X
- **Passos para Execução:**
 1. Tentar operação de divisão no serviço de cisterna.
- **Resultado Esperado:** Exceção `IllegalStateException`.
- **CrITÉrios de Sucesso:** Ação bloqueada antes da execução matemática.
- **CrITÉrios de Falha:** Erro oculto no sistema.

Módulo de Entregas de Água (Water Delivery)

52. Registro de Nova Entrega na API

- **ID do Caso de Teste:** CT-052
- **Nome:** Teste de Registro de Nova Entrega na API
- **Objetivo:** Validar a integração HTTP para criar um recibo de entrega.
- **Pré-condições:** Endpoint de entregas operante.
- **Entradas:**
 1. Payload de entrega (ID família, litragem recebida, litragem entregue)
- **Passos para Execução:**

1. Emitir POST para Endpoint de WaterDelivery.

- **Resultado Esperado:** Resposta 201 com o ID criado.
- **Critérios de Sucesso:** Criação de registro rastreável com os campos de litragem exatos.
- **Critérios de Falha:** Payload rejeitado (400) indevidamente.

53. Busca de Entregas por Ano e Família na API

- **ID do Caso de Teste:** CT-053
- **Nome:** Teste de Busca de Entregas por Ano e Família na API
- **Objetivo:** Prover histórico de atendimento a uma família no tempo.
- **Pré-condições:** Entrega no ano base existe.
- **Entradas:**

1. Ano base

2. ID Família

- **Passos para Execução:**

1. Fazer GET com as queries configuradas na API.

- **Resultado Esperado:** Lista HTTP 200 contendo as entregas solicitadas.
- **Critérios de Sucesso:** Dados expostos mostram que a busca ano/id reflete o banco.
- **Critérios de Falha:** Pesquisa expõe outras famílias/anos ou gera erro.

54. Gravação de Entrega de Água pelo Serviço

- **ID do Caso de Teste:** CT-054
- **Nome:** Teste de Gravação de Entrega de Água pelo Serviço
- **Objetivo:** Confirmar a persistência do serviço para uma entrega solicitada.
- **Pré-condições:** Repositório responde e os dados informados em DTO são válidos.
- **Entradas:**

1. WaterDeliveryDTO

• Passos para Execução:

1. Executar `saveWaterDelivery`.

- **Resultado Esperado:** Conversão do DTO para Model, save acionado, e retorno DTO com IDs.
- **CrITÉrios de Sucesso:** O repositório armazena a entrega fiel aos dados originais.
- **CrITÉrios de Falha:** Falha de Foreign Key no cadastro.

55. Cálculo de Desconto do Restante (Delivered Amount)

- **ID do Caso de Teste:** CT-055
- **Nome:** Teste de Cálculo de Desconto do Restante (Delivered Amount)
- **Objetivo:** Verificar se a entrega compensa sobras antes de fechar o registro final.
- **Pré-condições:** O volume distribuído excede a capacidade solicitada.
- **Entradas:**

1. Entregas e Capacidade da Cisterna base

• Passos para Execução:

1. Validar o cálculo de Delivered Amount no decorrer do save.

- **Resultado Esperado:** A quantidade entregue é registrada considerando os possíveis vazamentos de capacidade.
- **CrITÉrios de Sucesso:** O valor contabiliza exatamente o estocado na cisterna.
- **CrITÉrios de Falha:** Volumes perdidos não contabilizados na entrega gerando discrepância nos relatórios.

56. Consulta do Histórico Anual de Entrega

- **ID do Caso de Teste:** CT-056
- **Nome:** Teste de Consulta do Histórico Anual de Entrega
- **Objetivo:** Testar o filtro de dados na base via repositório de WaterDelivery.
- **Pré-condições:** Registros criados.

- **Entradas:**

1. Year: 2024

2. Family ID: 1L

- **Passos para Execução:**

1. Invocar ``findByYearAndFamilyId`` do Service.

- **Resultado Esperado:** Retorna a listagem temporal das entidades.

- **Critérios de Sucesso:** Os DTOs exibidos são do ano correspondente e da família certa.

- **Critérios de Falha:** Resultados misturados ou SQL divergente.

57. Consulta Vazia no Histórico de Entrega

- **ID do Caso de Teste:** CT-057

- **Nome:** Teste de Consulta Vazia no Histórico de Entrega

- **Objetivo:** Verificar comportamento de pesquisa quando nenhuma entrega atende os filtros.

- **Pré-condições:** Não existe histórico na base configurada.

- **Entradas:**

1. Year base

2. Family ID base

- **Passos para Execução:**

1. Invocar a função ``findByYearAndFamilyId``.

- **Resultado Esperado:** Deve retornar lista vazia.

- **Critérios de Sucesso:** O sistema não falha na ausência de registro e retorna uma lista vazia e imutável.

- **Critérios de Falha:** Null pointer ou exceção genérica.

Módulo de Configurações do Sistema

58. Recuperação de Configurações Existentes

- **ID do Caso de Teste:** CT-058
- **Nome:** Teste de Recuperação de Configurações Existentes
- **Objetivo:** Garantir o acesso às variáveis de ambiente de negócio globais.
- **Pré-condições:** O sistema possui registro de configurações inicializadas.
- **Entradas:**

1. Nenhuma

- **Passos para Execução:**

1. Acionar `getSystemSettings` do serviço de configuração.

- **Resultado Esperado:** O retorno do DTO do registro principal existente da tabela.
- **Critérios de Sucesso:** O ID e a litragem de consumo diário são entregues de forma precisa.
- **Critérios de Falha:** O sistema acusa registro inexistente.

59. Exceção na Recuperação por Falta de Configuração

- **ID do Caso de Teste:** CT-059
- **Nome:** Teste de Exceção na Recuperação por Falta de Configuração
- **Objetivo:** Confirmar se a ausência de definições aciona um comportamento previsto (e fatal).
- **Pré-condições:** Banco vazio para a tabela de configuração.
- **Entradas:**

1. Nenhuma

- **Passos para Execução:**

1. Rodar ``getSystemSettings`` em base isolada/mock vazio.

- **Resultado Esperado:** A aplicação deve emitir `EntityNotFoundException` indicando a anomalia.
- **Critérios de Sucesso:** O fluxo falha intencionalmente pois o sistema não pode funcionar sem configurações mínimas.
- **Critérios de Falha:** O sistema retorna instâncias fantasma com zero ou null e continua operando incorretamente.

60. Atualização Dinâmica do Consumo Diário

- **ID do Caso de Teste:** CT-060
- **Nome:** Teste de Atualização Dinâmica do Consumo Diário
- **Objetivo:** Permitir alteração na litragem projetada de consumo global do município por cidadão.
- **Pré-condições:** O painel administrativo enviou novos números.
- **Entradas:**

1. DTO contendo a nova litragem diária mínima estipulada.

- **Passos para Execução:**

1. Mapear, executar ``updateSystemSettings`` passando o Payload.

- **Resultado Esperado:** Atualização da base do sistema e retorno do novo modelo ajustado.
- **Critérios de Sucesso:** O Consumo projetado reflete a mudança imediatamente nas futuras rotinas.
- **Critérios de Falha:** A base congela na versão anterior ou falha na conversão de decimais.

61. Exceção na Atualização em Repositório Vazio

- **ID do Caso de Teste:** CT-061
- **Nome:** Teste de Exceção na Atualização em Repositório Vazio
- **Objetivo:** Testar a resiliência na alteração sem o registro âncora presente.
- **Pré-condições:** Update tentado antes da primeira instalação completa do ambiente de negócio.

- **Entradas:**

1. DTO da atualização

- **Passos para Execução:**

1. Executar ``updateSystemSettings``.

- **Resultado Esperado:** EntityNotFoundException na tentativa de encontrar a configuração âncora.

- **Critérios de Sucesso:** Bloqueia gravações fantasma e avisa o erro gravíssimo.

- **Critérios de Falha:** O sistema insere um segundo registro global e bagunça lógicas estatísticas.